

ЛАБОРАТОРНАЯ РАБОТА № 06

«Введение в многопоточность»

Группа: РИМ-150950

Студент: Ляхова Елизавета Олеговна

Ссылка на github: https://github.com/Lizaswer/java_course_master.git

Цель: получить представление о механизме многопоточности в языке
программирования Java

Ход работы:

1 Напишите программу, которая запускает два потока и выводит на экран их имена и текущее время в течение 10 секунд.

```
package lrl6;

public class task1 {
    public static void main(String[] args) throws
        InterruptedException {
        Thread t1 = new Thread(() -> {
            long end = System.currentTimeMillis() + 10000;
            while (System.currentTimeMillis() < end) {

                System.out.println(Thread.currentThread().getName() +
                    " - " + new java.util.Date());
                try { Thread.sleep(1000); } catch
                    (InterruptedException e) {}
            }
        });

        Thread t2 = new Thread(() -> {
            long end = System.currentTimeMillis() + 10000;
            while (System.currentTimeMillis() < end) {

                System.out.println(Thread.currentThread().getName() +
                    " - " + new java.util.Date());
                try { Thread.sleep(1000); } catch
                    (InterruptedException e) {}
            }
        });

        t1.start(); t2.start();
        t1.join(); t2.join();
    }
}
```

2 Напишите программу, которая запускает поток и выводит на экран числа от 1 до 10 с задержкой в 1 секунду между выводами.

```
package lrl6;

public class task2 {
    public static void main(String[] args) throws
    InterruptedException {
        Thread t = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                System.out.println(i);
                try { Thread.sleep(1000); } catch
                (InterruptedException e) {}
            }
        });
        t.start();
        t.join();
    }
}
```

3 Напишите программу, которая создает два потока, которые выводят на экран числа от 1 до 10 Первый поток должен выводить только четные числа, а второй поток должен выводить только нечетные числа.

```
package lrl6;

public class task3 {
    private static int number = 1;
    private static final Object lock = new Object();

    public static void main(String[] args) {
        Thread even = new Thread(() -> {
            synchronized (lock) {
                while (number <= 10) {
                    if (number % 2 == 0) {
                        System.out.println("Even: " +
                        number++);
                    }
                }
            }
        });
    }
}
```

```

        lock.notifyAll();
    } else {
        try { lock.wait(); } catch
(InterruptedOperationException e) {}
    }
}

});
Thread odd = new Thread(() -> {
    synchronized (lock) {
        while (number <= 10) {
            if (number % 2 != 0) {
                System.out.println("Odd: " +
number++);

                lock.notifyAll();
            } else {
                try { lock.wait(); } catch
(InterruptedOperationException e) {}
            }
        }
    }
});
even.start(); odd.start();
}
}

```

4 Напишите программу, которая создает 10 потоков и каждый поток выводит на экран свой номер.

```

package lrl6;

public class task4 {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            final int num = i;
            new Thread(() -> System.out.println("Я
поток №" + num)).start();

```

```
}  
}  
}
```

5 Напишите функцию, которая находит максимальный элемент в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.

```
package lrl6;  
import java.util.concurrent.*;  
  
public class task5 {  
  
    public static void main(String[] args) throws  
        Exception {  
        int[] array = {3, 7, 2, 9, 1, 10, 4, 8, 5, 6};  
        int cores =  
            Runtime.getRuntime().availableProcessors();  
        ExecutorService executor =  
            Executors.newFixedThreadPool(cores);  
        int chunkSize = (int) Math.ceil(array.length /  
            (double) cores);  
        Future<Integer>[] futures = new Future[cores];  
  
        for (int i = 0; i < cores; i++) {  
            final int start = i * chunkSize;  
            final int end = Math.min(start + chunkSize,  
                array.length);  
            futures[i] = executor.submit(() -> {  
                int max = Integer.MIN_VALUE;  
                for (int j = start; j < end; j++) max =  
                    Math.max(max, array[j]);  
                return max;  
            });  
        }  
        int globalMax = Integer.MIN_VALUE;  
        for (Future<Integer> f : futures) globalMax =  
            Math.max(globalMax, f.get());  
    }  
}
```

```

        executor.shutdown();
        System.out.println("Максимум: " + globalMax);
    }
}

```

6 Напишите функцию, которая суммирует элементы в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.

```

package lrl6;
import java.util.concurrent.*;

public class task6 {

    public static void main(String[] args) throws
    Exception {
        int[] array = {1,2,3,4,5,6,7,8,9,10};
        int cores =
        Runtime.getRuntime().availableProcessors();
        ExecutorService executor =
        Executors.newFixedThreadPool(cores);
        int chunkSize = (int) Math.ceil(array.length /
        (double) cores);
        Future<Integer>[] futures = new Future[cores];

        for (int i = 0; i < cores; i++) {
            final int start = i * chunkSize;
            final int end = Math.min(start + chunkSize,
            array.length);
            futures[i] = executor.submit(() -> {
                int sum = 0;
                for (int j = start; j < end; j++) sum +=
            array[j];
                return sum;
            });
        }
        int totalSum = 0;
    }
}

```

```

        for (Future<Integer> f : futures) totalSum +=
f.get();
        executor.shutdown();
        System.out.println("Сумма: " + totalSum);
    }
}

```

7 Решите две любых задачи с сайта timus.

Задача 1401

```

package lrl6.timus;

import java.util.Scanner;

public class task1401 {
    private static int[][] board;
    private static int tileNumber = 1;
    private static int n;
    private static int size;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        int x = sc.nextInt();
        int y = sc.nextInt();
        size = 1 << n; // 2^n
        board = new int[size + 1][size + 1];

        board[x][y] = 0;
        tile(1, 1, size, x, y);
        printBoard();
    }

    private static void tile(int row, int col, int
size, int missingRow, int missingCol) {
        if (size == 1) return;

```

```

    int half = size / 2;
    int currentTile = tileNumber++;

    int missingQuadrant = getQuadrant(row, col,
half, missingRow, missingCol);

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            int quad = i * 2 + j + 1;
            if (quad != missingQuadrant) {
                int centerRow = row + i * half +
half - 1;

                int centerCol = col + j * half +
half - 1;

                board[centerRow][centerCol] =
currentTile;
            }
        }
    }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            int quad = i * 2 + j + 1;
            int newRow = row + i * half;
            int newCol = col + j * half;
            int newMissingRow, newMissingCol;

            if (quad == missingQuadrant) {

                newMissingRow = missingRow;
                newMissingCol = missingCol;
            } else {

                newMissingRow = row + i * half +
half - 1;

                newMissingCol = col + j * half +
half - 1;

```



```

        }
        tile(newRow, newCol, half,
newMissingRow, newMissingCol);
    }
}

private static int getQuadrant(int row, int col,
int half, int missingRow, int missingCol) {
    if (missingRow < row + half) {
        if (missingCol < col + half) return 1;
        else return 2;
    } else {
        if (missingCol < col + half) return 3;
        else return 4;
    }
}

private static void printBoard() {
    for (int i = 1; i <= size; i++) {
        for (int j = 1; j <= size; j++) {
            System.out.print(board[i][j] + (j ==
size ? "" : " "));
        }
        System.out.println();
    }
}
}

```

Задача 1402

```

package lrl6.timus;

import java.math.BigInteger;
import java.util.Scanner;

public class task1402 {

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();

    BigInteger fact = BigInteger.ONE;
    for (int i = 2; i <= n; i++) {
        fact =
fact.multiply(BigInteger.valueOf(i));
    }

    BigInteger sum = BigInteger.ZERO;
    for (int k = 2; k <= n; k++) {
        // A(n,k) = n! / (n-k)!
        BigInteger permutations =
fact.divide(fact(n - k));
        sum = sum.add(permutations);
    }

    System.out.println(sum);
}

private static BigInteger fact(int m) {
    BigInteger result = BigInteger.ONE;
    for (int i = 2; i <= m; i++) {
        result =
result.multiply(BigInteger.valueOf(i));
    }
    return result;
}
}

```

Вывод: В ходе выполнения данной лабораторной работы были изучены и практически освоены основные механизмы многопоточного программирования в Java. Были решены задачи различного уровня

сложности, что позволило получить полное представление о возможностях и особенностях работы с потоками.